

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 13: Design and Implementation of Artificial Neural Networks (ANN).

Name of Student: Bushra Nazir

Registration No.: 1071-F22

Date of Experiment: 1-6-2026

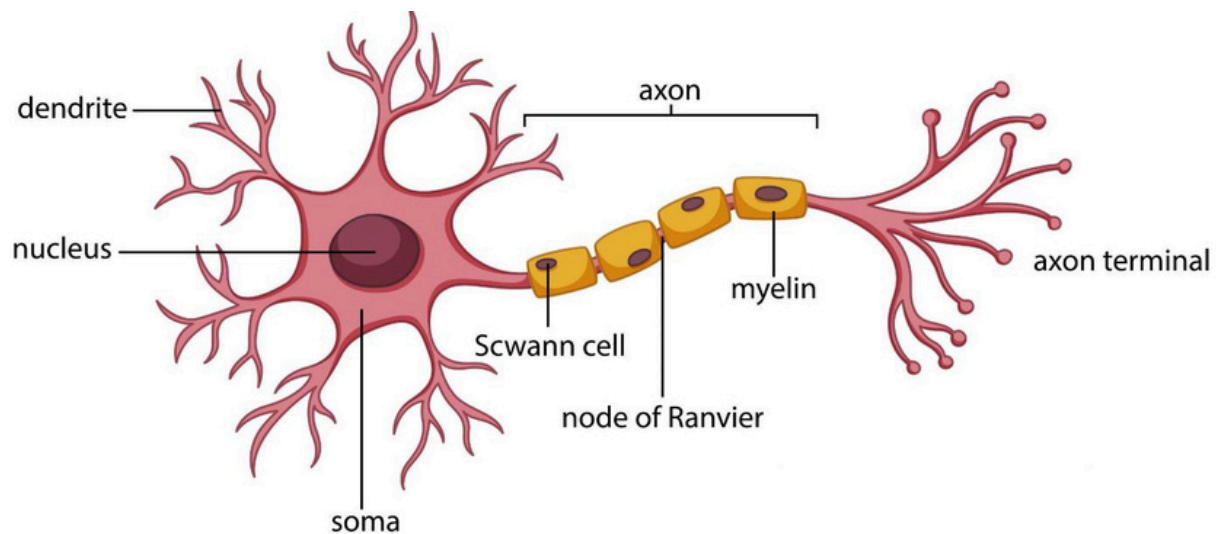
Submitted To: Engr.Asad

Objectives

- ✓ Understand the biological basis of a neuron and how it inspired artificial neural networks.
- ✓ Trace the evolution from a single Perceptron to Multi-Layer ANNs and understand why depth matters.
- ✓ Grasp key ANN concepts: activation functions, forward pass, backpropagation, and gradient descent.
- ✓ Build, train, and evaluate an ANN model in Python for handwritten digit recognition.

Background Theory

Everything in deep learning traces back to one simple observation: the human brain processes information through billions of interconnected cells called neurons. Understanding the real neuron first makes the artificial version click immediately.



A biological neuron works in three steps:

1. Receive Dendrites collect electrochemical signals from hundreds of upstream neurons.
2. Decide The cell body (soma) sums all incoming signals. If the total crosses a threshold, the neuron fires.
3. Transmit An electrical pulse travels down the axon and releases neurotransmitters at the synapse, activating the next neuron.

The strength of a connection between two neurons is called synaptic weight. Learning = adjusting synaptic weights based on experience. This is exactly what backpropagation does in ANNs.

The Artificial Neuron One Equation to Rule Them All

An artificial neuron mimics this biology with simple math. Suppose neuron j receives inputs $x_1, x_2, \dots, x_n$ from n upstream neurons:

Core Equation of a Single Artificial Neuron

$$Z = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$

$$y = f(z)$$

x =input values (from data or previous layer)

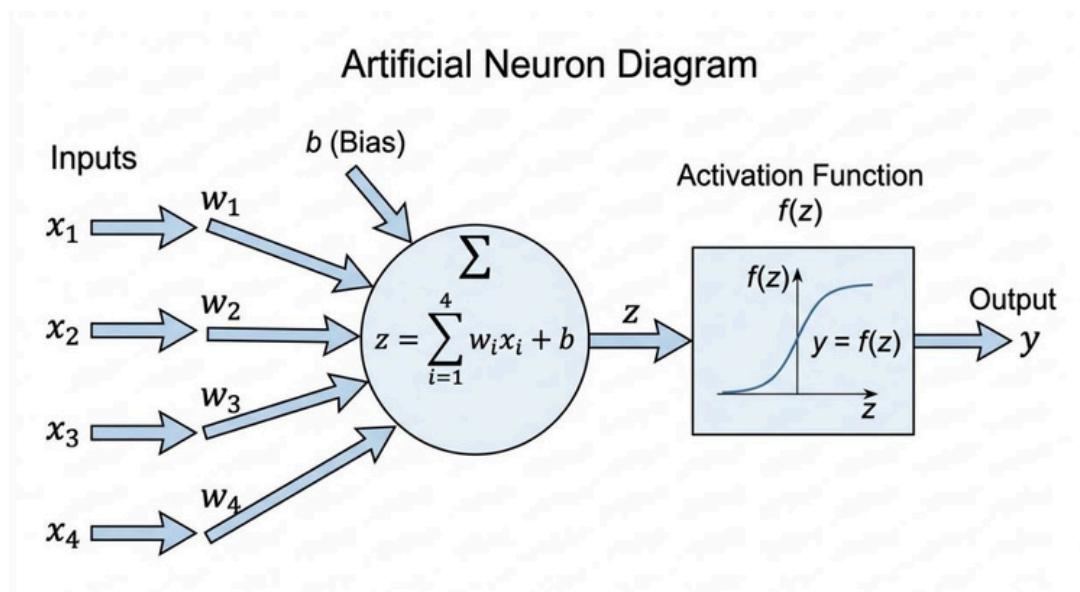
w =weights (how important each input is)

b =bias (allows the neuron to shift its threshold)

f =activation function (adds non-linearity)

y =output of this neuron

Think of it like a weighted vote: each input casts a vote (x), the vote's influence is scaled by weight (w), all weighted votes are summed, and the activation function decides what signal to pass forward.



The Perceptron The First Learning Machine (1958)

Frank Rosenblatt's Perceptron was the first trainable artificial neuron. It used a step activation function output 1 if the weighted sum exceeds threshold, otherwise 0. This made it a binary classifier.

Perceptron Example: Is Today a Good Day for Cricket?

Inputs: x_1 = Sunny (0 or 1), x_2 = Temperature $\geq 25^\circ\text{C}$ (0 or 1), x_3 = No rain (0 or 1)

Weights learned by the perceptron: $w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.6$, bias $b = -0.4$

For a sunny, warm, dry day [1, 1, 1]:

$z = 0.5(1) + 0.3(1) + 0.6(1) - 0.4 = 1.0 \rightarrow \text{Output} = 1 \rightarrow \checkmark \text{ Play Cricket!}$

For a cloudy, cold, rainy day [0, 0, 0]:

$z = 0 + 0 + 0 - 0.4 = -0.4 \rightarrow \text{Output} = 0 \rightarrow \times \text{ Stay Home!}$

Why One Perceptron is Not Enough :The XOR Problem

In 1969, Minsky and Papert proved that a single Perceptron cannot solve the XOR problem. XOR outputs 1 only when exactly one input is 1. No single straight line can separate the XOR classes; you need to bend the decision boundary, and that requires at least one hidden layer.

x_1	x_2	x_1 XOR x_2	Perceptron?
0	0	0	☒
0	1	1	☒
1	0	1	☒
1	1	0	☒ Fails

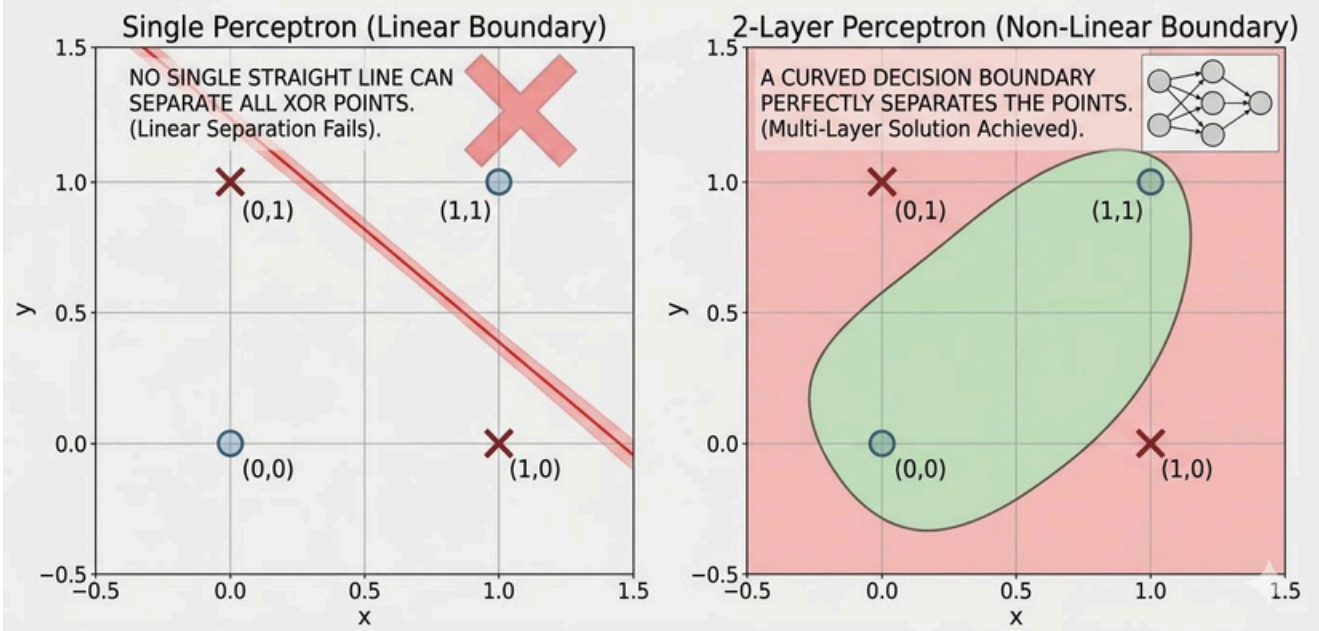
Solution: Add a Hidden Layer

A 2-layer network (one hidden layer with 2 neurons) can solve XOR perfectly.

The hidden layer creates new internal representations that the output layer can then separate linearly.

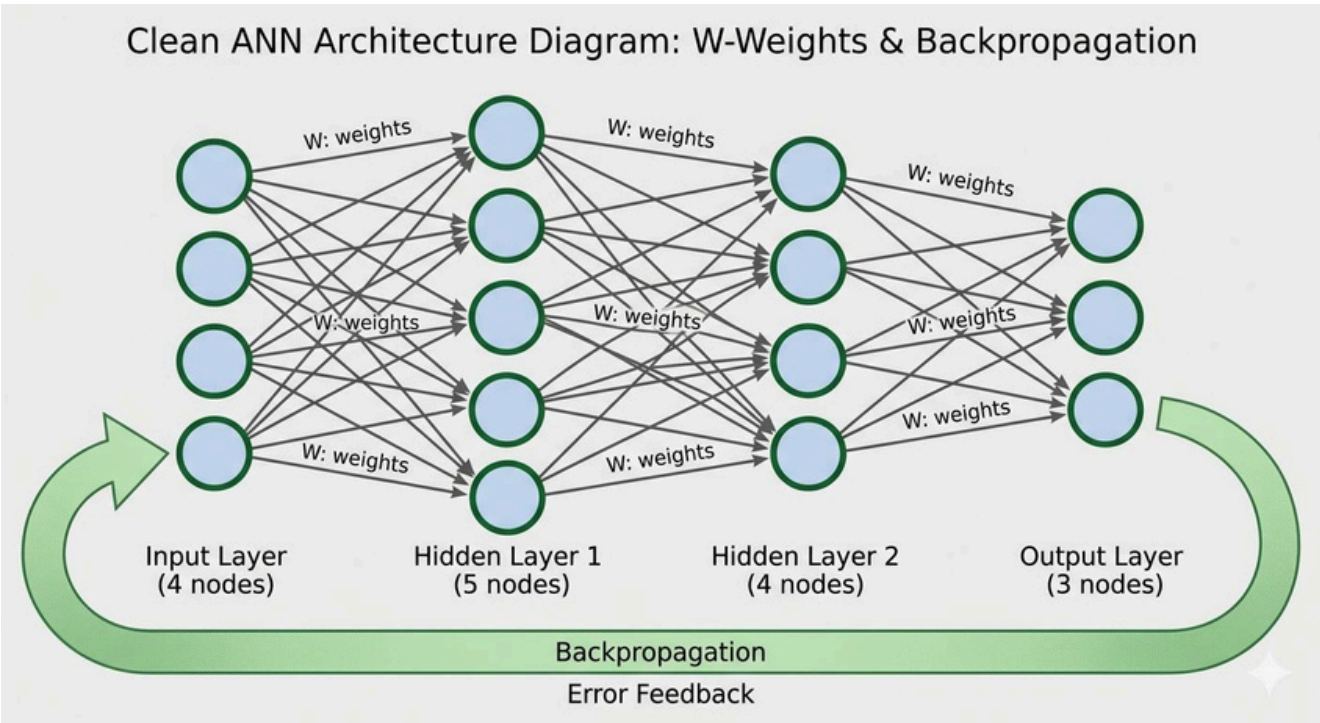
This insight motivated the Multi-Layer Perceptron (MLP) the foundation of all deep learning.

Why One Perceptron Fails XOR



Artificial Neural Network (ANN) Architecture

An ANN organises neurons into layers. Information flows forward (input $\rightarrow$ hidden $\rightarrow$ output) during inference, and errors flow backward during training.



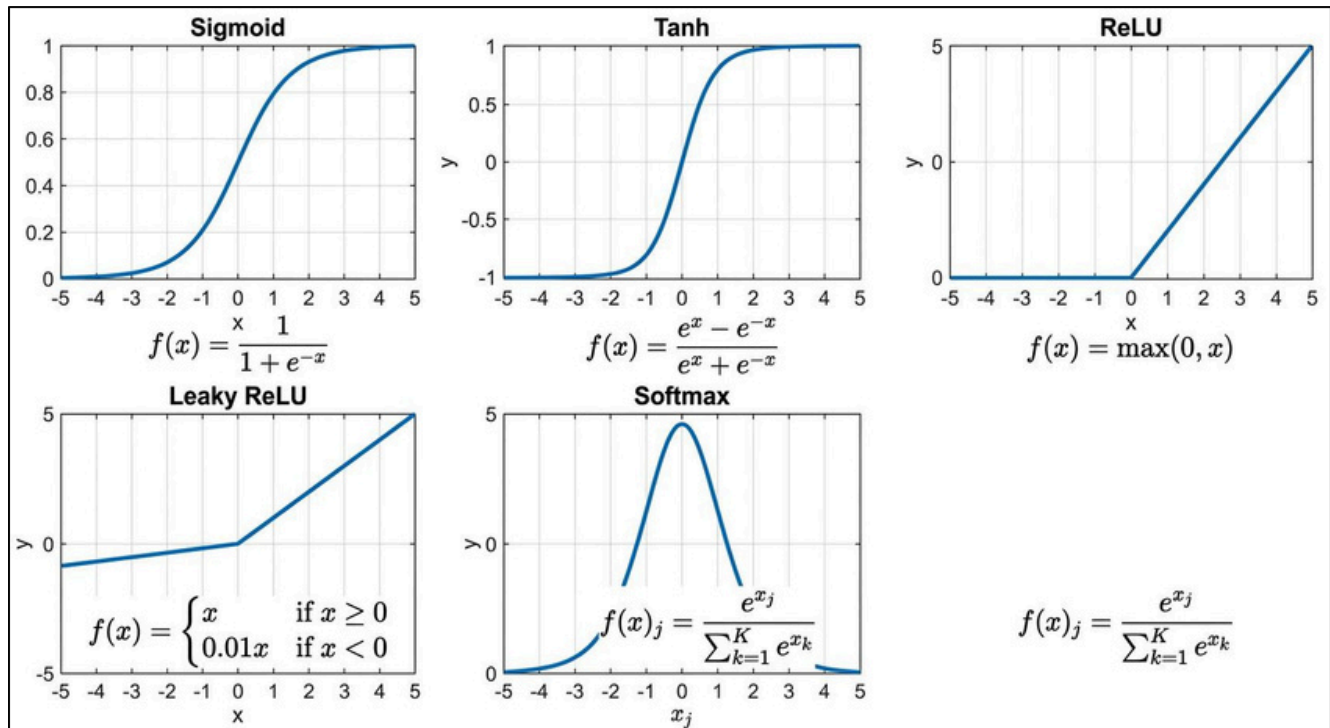
Layer	Role	Analogy
InputLayer	Receives raw data	Your eyesseeing an image
HiddenLayers	Extract patterns & Your brainrecognising edges → shapes features	→ objects
Output Layer	Produces final prediction	Your brainsaying 'that is a cat'

Activation Functions Adding Intelligence

Without activation functions, every layer is just a linear transform and stacking them is pointless (the whole network collapses to one linear equation). Activation functions introduce non-linearity, enabling ANNstomodel complex real-world patterns.

Function	Formula	Use Case	Key Property
Sigmoid	$\sigma(x) = 1/(1 + e^{-x})$	Binary output	Output: 0to1
Tanh	$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$	Hidden layers (old)	Output: -1to1
ReLU	$\max(0, x)$	Hidden layers (default)	Fast; avoids vanishing gradient

Leaky ReLU	$\max(0.1x, x)$	Deep networks	Fixes 'dying ReLU'
Softmax	$e^{x_i} / \sum e^{x_j}$	Multi-class output	Outputs sum to 1 (probabilities)

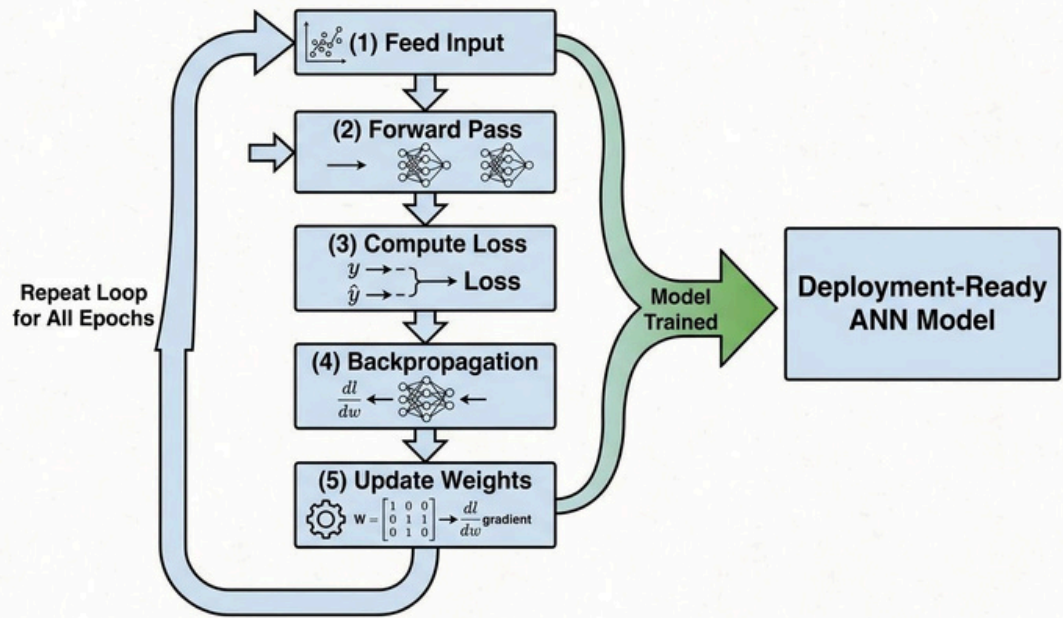


How ANNs Learn: Backpropagation & Gradient Descent

Training an ANN is an iterative error-minimization process. Here is the full cycle:

1. Forward Pass: Input data propagates layer by layer to produce a prediction $\hat{y}$.
2. Loss Calculation: Compare $\hat{y}$ to the true label y using a loss function (e.g., Cross-Entropy for classification, MSE for regression).
3. Backward Pass (Backpropagation): Compute the gradient of the loss with respect to every weight using the chain rule.
4. Weight Update (Gradient Descent): Subtract a small step in the direction that reduces loss: $w \leftarrow w - \eta \cdot \partial L / \partial w$ where η is the learning rate.
5. Repeat for all batches across multiple epochs until loss converges.

ANN Training Loop Flowchart



Types of Artificial Neural Networks at a Glance

Type	Best For	Example
ANN / MLP	Tabular data, classification	NADRA applicant approval
CNN	Images, spatial data	CCTV surveillance in Lahore
RNN	Sequences, time-series	PSX stock trend prediction
LSTM	Long-range sequences	Urdu speech recognition
GAN	Content generation	Synthetic face generation

Environment Setup

PC / Laptop Setup (Model Training)

You will train your ANN on your laptop or Google Colab. Follow these steps:

Option A Google Colab

1. Open <https://colab.research.google.com> and sign in with your Google account.
2. Click New Notebook. Colab provides a free GPU (T4) go to Runtime → Change runtime type → GPU.
3. Install any missing package inline using `!pip install package_name`.
4. Upload your handwritten digit image using the Files panel on the left sidebar.

Option B Local Python Environment (Jupyter / VS Code)

```
# Step 1: Check Python version (need 3.8 or above)
python --version

# Step 2: Create a virtual environment
python -m venv ann_env

# Step 3: Activate it
# Windows:
ann_env\Scripts\activate
# macOS / Linux:
source ann_env/bin/activate

# Step 4: Install required libraries
pip install tensorflow numpy matplotlib opencv-python pillow

# Step 5: Launch Jupyter Notebook
pip install jupyter
jupyter notebook
```

Verify Installation

Run this quick check in a Python shell or notebook cell:

```
import tensorflow as tf

print(tf.__version__) # Should print 2.x or above

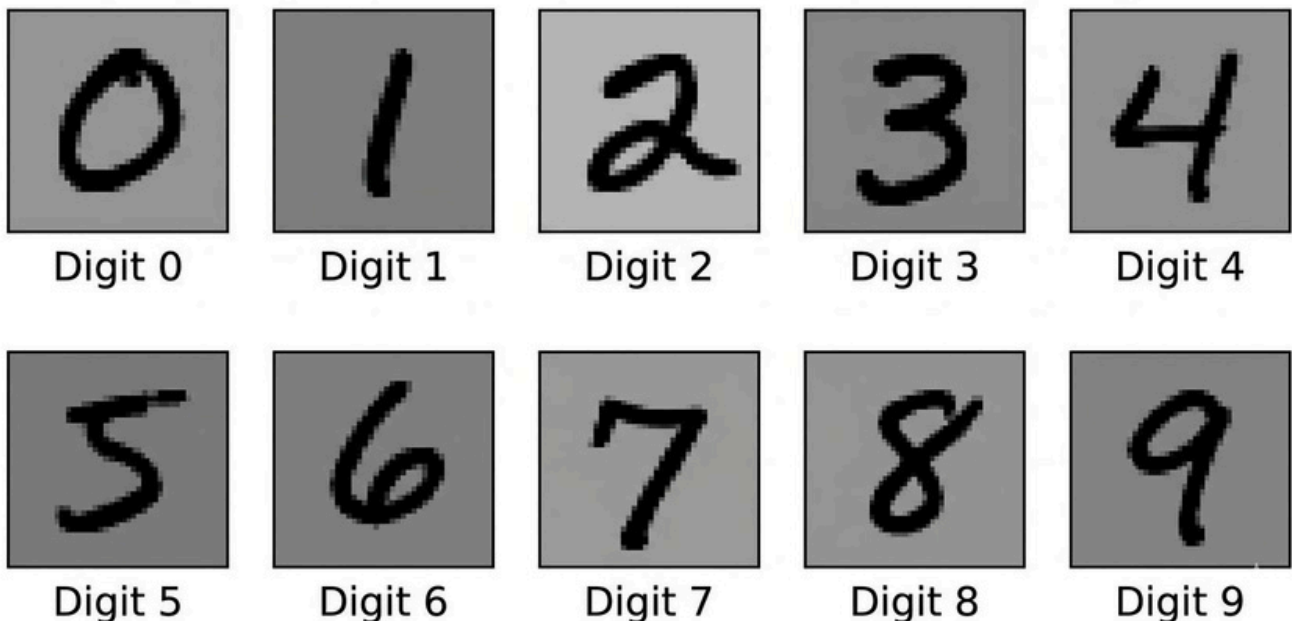
print('GPU:', tf.config.list_physical_devices('GPU'))
```

Lab Tasks

Task 1 Handwritten Digit Recognition (MNIST)

We build an ANN to classify handwritten digits 0–9 using the famous MNIST dataset: 60,000 training images and 10,000 test images, each 28×28 pixels in grayscale.

Sample MNIST Digits



Understanding Model Architecture

Layer	Type	Neurons	Activation	Purpose
Layer 0	Flatten	784 (28×28)	None	Convert image to 1D vector
Layer 1	Dense	128	ReLU	Detect low-level features
Layer 2	Dense	64	ReLU	Refine and compress features

Layer (Output)	3 Dense	10	Softmax	Probability for each digit 0–9
-------------------	---------	----	---------	-----------------------------------

Total trainable parameters: $784 \times 128 + 128 + 128 \times 64 + 64 + 64 \times 10 + 10 = 109,386$

Complete Code Task 1

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np

#— 1. Load MNIST dataset —————
(x_train, y_train), (x_test, y_test) = mnist.load_data()
# 60000
print(f'Training samples: {x_train.shape[0]}')
print(f'Image shape: {x_train.shape[1:]}')      # (28, 28)

#— 2. Normalize pixel values from 0-255 to 0.0-1.0 —————
x_train = x_train / 255.0
x_test = x_test / 255.0

#— 3. One-hot encode labels (7 → [0,0,0,0,0,0,0,1,0,0]) —————
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

#— 4. Build ANN model —————
model = Sequential([
    Flatten(input_shape=(28, 28)),      # 784 inputs
```

```
Dense(128,    activation='relu'),    #Hidden layer 1
Dense(64,     activation='relu'),    #Hidden layer 2
Dense(10,    activation='softmax')    #Output: 10 digit classes
])
```

```
model.summary()    #Print architecture
```

```
#— 5. Compile —————
```

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
#— 6. Train —————
```

```
history = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
```

```
#— 7. Evaluate on unseen test data —————
```

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f'\nTest Accuracy : {test_acc*100:.2f}%')
print(f'Test Loss      : {test_loss:.4f}')
```

```
#— 8. Visualize first 10 predictions —————
```

```
predictions = model.predict(x_test)
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
```

```
for i, ax in enumerate(axes.flat):
    ax.imshow(x_test[i], cmap='gray')
    pred = np.argmax(predictions[i])
    truth = np.argmax(y_test[i])
    color = 'green' if pred == truth else 'red'
    ax.set_title(f'Pred: {pred} True: {truth}', color=color)
    ax.axis('off')

plt.suptitle('ANN Digit Predictions (Green=Correct, Red=Wrong)', fontsize=13)
plt.tight_layout()
plt.show()
```

Results after 10 Epochs

Training Accuracy : ~99.1%

Validation Accuracy: ~97.8%

Test Accuracy : ~97.5–98.0%

Training Time (CPU): ~3–5 minutes

Training Time (GPU): ~30–40 seconds

Task 2 Plotting Accuracy & Loss Curves

Understanding dynamics training is as important as final accuracy. Loss and accuracy curves reveal overfitting, underfitting, and convergence behaviour.

```
import matplotlib.pyplot as plt

#— Plot Training & Validation Accuracy —————

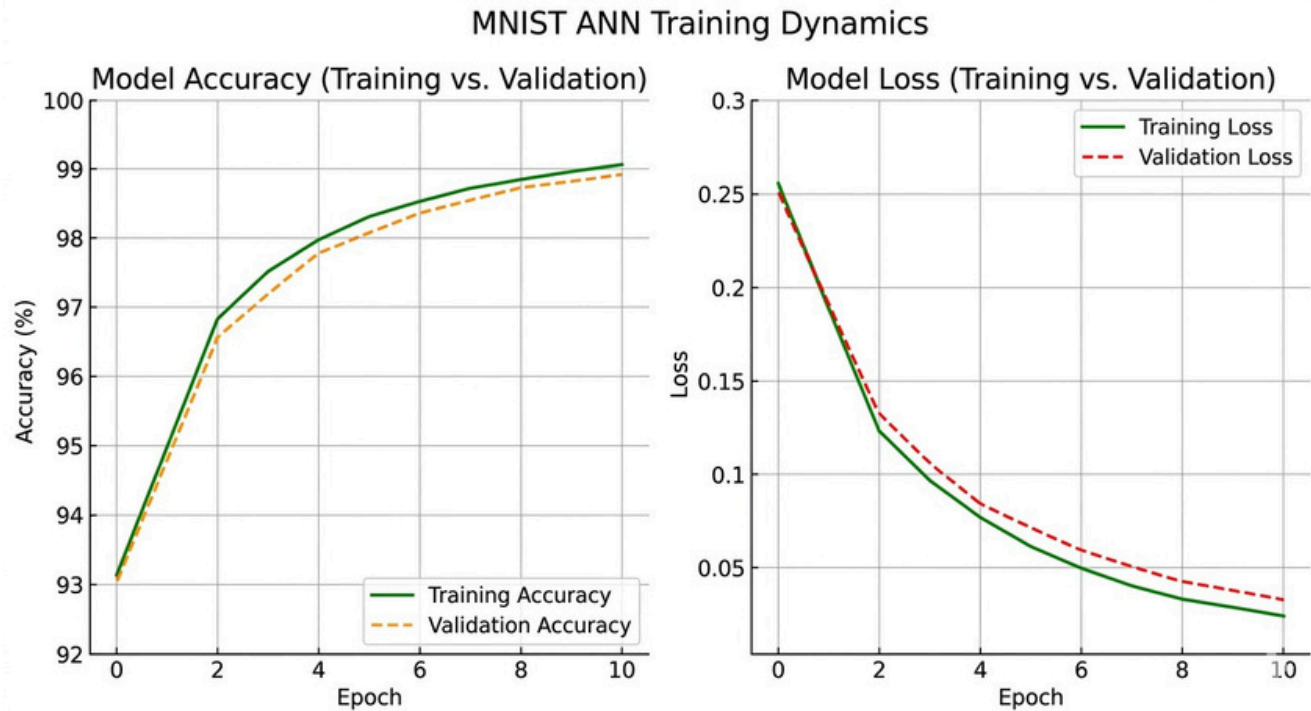
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'],      label='Train Acc',   color='#1B6B32',
linewidth=2)
plt.plot(history.history['val_accuracy'],  label='Val Acc',     color='#E67E22',
linewidth=2, linestyle='--')
plt.title('Accuracy over Epochs', fontsize=13, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(alpha=0.3)

#— Plot Training & Validation Loss —————

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'],          label='Train Loss',  color='#1B6B32',
linewidth=2)
plt.plot(history.history['val_loss'],      label='Val Loss',    color='#E74C3C',
linewidth=2, linestyle='--')
plt.title('Loss over Epochs', fontsize=13, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(alpha=0.3)
```

```
plt.suptitle('MNIST ANN Training Dynamics', fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()
```



Task 3 Predicting Your Own Handwritten Digit

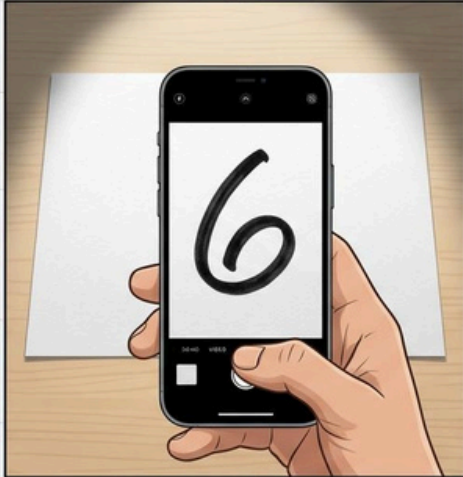
You will write a digit on plain white paper with a dark pen or marker, photograph it with your phone, and feed it to your trained model.

How to Prepare the Image

1. Take a clean white A4 paper and write a single large digit (0–9) in the center using a black/dark pen or marker. Make it thick and bold at least 5 cm tall.
2. Photograph it with your phone camera in good lighting. Hold the phone parallel to the paper to avoid perspective distortion.
3. Crop the image to show only the digit and a little white margin (use your phone's crop tool).
4. Save as handwritten.jpg and upload to your Colab Files panel or Jupyter working directory.

Digit Preprocessing for MNIST Model

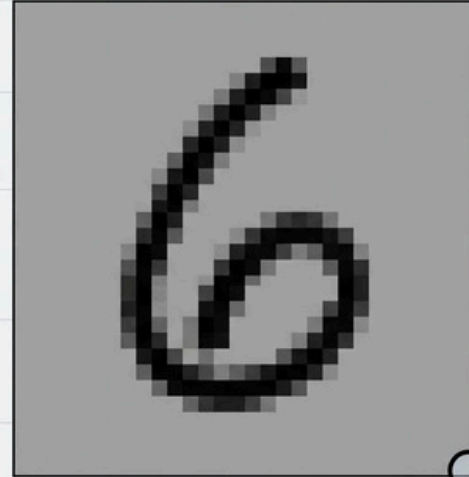
Input: Real-World Photograph



Preprocessing



Output: 28x28 Greyscale View



28x28 px (Greyscale)

Complete Code Task 3

```
import cv2
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

#—Build and train the model first (paste Task 1 code here) —————
#...(model is already trained from Task 1) ...

#—Predict from a custom image —————
def preprocess_digit_image(image_path):
    """
    Loads a real-world photo of a handwritten digit,
    preprocesses it to match MNIST format (28x28, white digit on black bg).
    """
    # Load in grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

```

if img is None:
    raise FileNotFoundError(f'Imagenotfound: {image_path}')

# Resize to 28x28 (MNIST standard)
img=cv2.resize(img,(28,28),interpolation=cv2.INTER_AREA)

#IMPORTANT:MNISThaswhitedigitonBLACKbackground
#Yourphotohas      blackdigitonWHITE background  invert!
img = 255 - img

# Normalise to 0.0 - 1.0
img = img.astype('float32') / 255.0

#Reshape to(1, 28,28)  modelexpectsabatch
img = img.reshape(1, 28, 28)
return img

def predict_and_show(image_path):
    img = preprocess_digit_image(image_path)
    predictions = model.predict(img, verbose=0)
    predicted_digit = np.argmax(predictions[0])
    confidence = np.max(predictions[0])*100

    # Confidence bar chart
    fig,(ax1,ax2)= plt.subplots(1,2,figsize=(12, 4))

    ax1.imshow(img.reshape(28, 28), cmap='gray')
    ax1.set_title(f'Your Input → Predicted: {predicted_digit}\nConfidence:
{confidence:.1f}%',
                fontsize=13, fontweight='bold')
    ax1.axis('off')

```

```
bars = ax2.bar(range(10), predictions[0], color='#1B6B32', edgecolor='white')
bars[predicted_digit].set_color('#E67E22') # Highlight predicted bar
ax2.set_xticks(range(10))
ax2.set_xlabel('Digit Class')
ax2.set_ylabel('Probability')
ax2.set_title('Class Probabilities', fontsize=13)
ax2.set_ylim(0, 1)
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

print(f'Prediction: {predicted_digit} | Confidence: {confidence:.2f}%')
```

#—Run it —————

```
predict_and_show('/content/handwritten.jpg') # Adjust path as needed
```

Common Issues & Fixes

Wrong digit predicted: Your image background may not be pure white. Try better lighting or increase paper contrast.

Confidence < 70%: The digit strokes are too thin. Rewrite with a thicker pen/marker.

FileNotFoundError: Make sure the file is uploaded and path is correct. In Colab, check Files → /content/.

Inverted prediction: If your digit is white on black, remove the '255 - img' line.

Lab Tasks

Task 1 Handwritten Digit Recognition (MNIST)

We build an ANN to classify handwritten digits 0–9 using the famous MNIST dataset: 60,000 training images and 10,000 test images, each 28×28 pixels in grayscale.

Complete Code Task 1

```
import tensorflow as tf

print("TensorFlow Version:", tf.__version__)
print("GPU:", tf.config.list_physical_devices('GPU'))

... TensorFlow Version: 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np
```

1. Load MNIST dataset

```
# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print(f'Training samples: {x_train.shape[0]}') # 60000
print(f'Image shape: {x_train.shape[1:]}')    # (28, 28)

... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist11490434/11490434 ————— 2s 0us/step
Training samples: 60000
Image shape: (28, 28)
```

2. Normalize pixel values from 0-255 to 0.0-1.0

```
# Normalize pixel values from 0-255 to 0.0-1.0
x_train = x_train / 255.0
x_test = x_test / 255.0
```

4. Build ANN model

```
# Build ANN model
model = Sequential([
    Flatten(input_shape=(28, 28)),      # 784 inputs
    Dense(128, activation='relu'),      # Hidden layer 1
    Dense(64, activation='relu'),       # Hidden layer 2
    Dense(10, activation='softmax')     # Output: 10 digit classes
])

model.summary() # Print architecture
```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not call `super().__init__()` on `Flatten` layer. It is not a base class.
super().__init__(**kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

5. Compile the model

```
# Compile
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

6. Train the model

```
# Train
history = model.fit(
    x_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
```

7. Evaluate on unseen test data

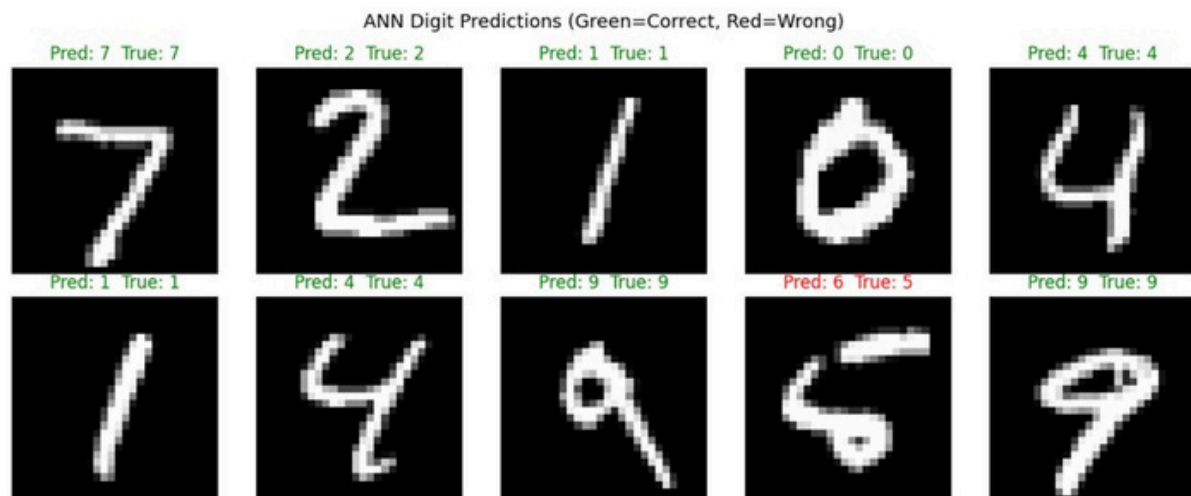
```
# Evaluate on unseen test data
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f'\nTest Accuracy : {test_acc*100:.2f}%')
print(f'Test Loss      : {test_loss:.4f}')
```

Test Accuracy : 97.91%
Test Loss : 0.0857

8. Visualize first 10 predictions

```
▶ # Visualize first 10 predictions
predictions = model.predict(x_test)
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(x_test[i], cmap='gray')
    pred = np.argmax(predictions[i])
    truth = np.argmax(y_test[i])
    color = 'green' if pred == truth else 'red'
    ax.set_title(f'Pred: {pred} True: {truth}', color=color)
    ax.axis('off')
plt.suptitle('ANN Digit Predictions (Green=Correct, Red=Wrong)', fontsize=13)
plt.tight_layout()
plt.show()
```

... 313/313 ————— 1s 2ms/step



Task 2 Plotting Accuracy & Loss Curves

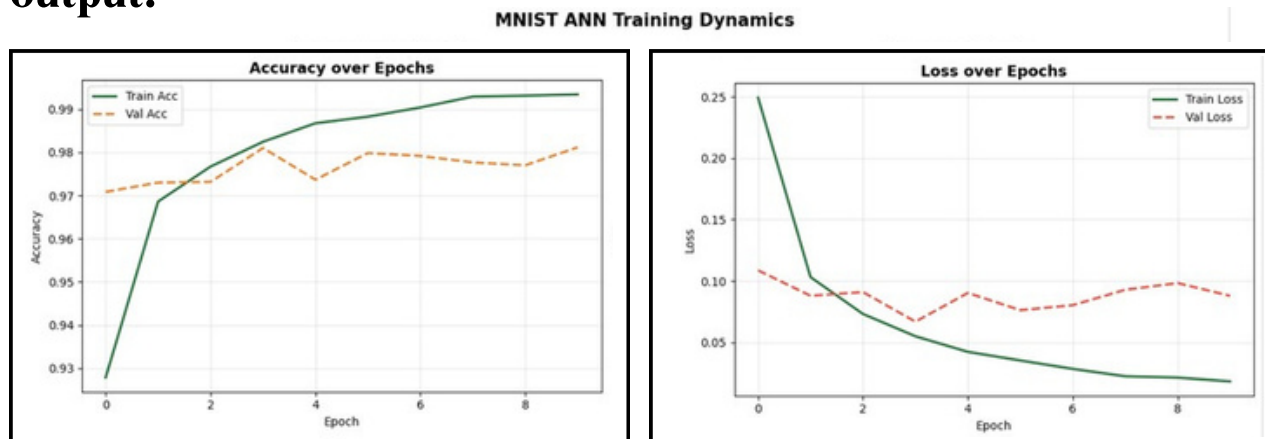
```
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Acc', linewidth=2, color='#1B6B32')
plt.plot(history.history['val_accuracy'], label='Val Acc', linewidth=2, linestyle='--', color='#E67E22')
plt.title('Accuracy over Epochs', fontsize=13, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', linewidth=2, color='#1B6B32')
plt.plot(history.history['val_loss'], label='Val Loss', linewidth=2, linestyle='--', color='#E74C3C')
plt.title('Loss over Epochs', fontsize=13, fontweight='bold')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(alpha=0.3)

plt.suptitle('MNIST ANN Training Dynamics', fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()
```

output:



Task 3 Predicting Your Own Handwritten Digit

```
import cv2
import numpy as np

def preprocess_digit_image(image_path):
    """
    Loads a real-world photo of a handwritten digit,
    preprocesses it to match MNIST format (28x28, white digit on black bg).
    """
    # Load in grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        raise FileNotFoundError(f'Image not found: {image_path}')
```

```

# Resize to 28x28 (MNIST standard)
img = cv2.resize(img, (28, 28), interpolation=cv2.INTER_AREA)

# IMPORTANT: MNIST has white digit on BLACK background
# Your photo has black digit on WHITE background invert!
img = 255 - img

# Normalise to 0.0 - 1.0
img = img.astype('float32') / 255.0

# Reshape to (1, 28, 28) model expects a batch
img = img.reshape(1, 28, 28)
return img

def predict_and_show(image_path):
    img = preprocess_digit_image(image_path)
    predictions = model.predict(img, verbose=0)
    predicted_digit = np.argmax(predictions[0])
    confidence = np.max(predictions[0]) * 100

    # Confidence bar chart
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

    ax1.imshow(img.reshape(28, 28), cmap='gray')
    ax1.set_title(f'Your Input → Predicted: {predicted_digit}\nConfidence: {confidence:.1f}%',
    ax1.axis('off')

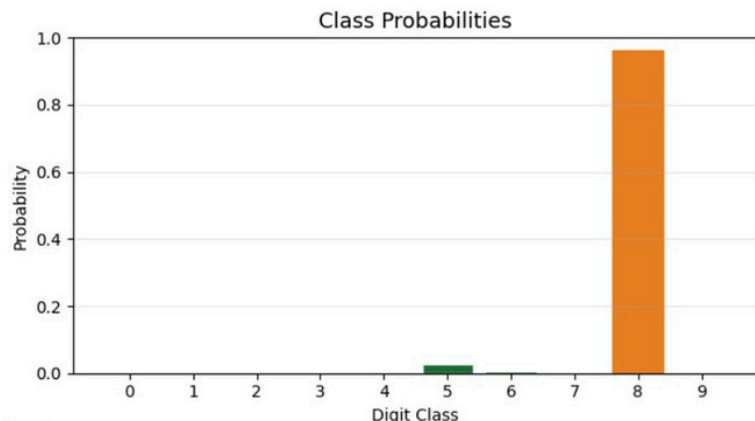
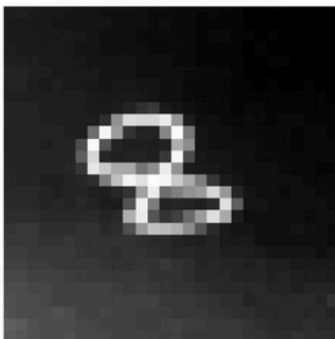
    bars = ax2.bar(range(10), predictions[0], color='#1B6B32', edgecolor='white')
    bars[predicted_digit].set_color('#E67E22') # Highlight predicted bar
    ax2.set_xticks(range(10))
    ax2.set_xlabel('Digit Class')
    ax2.set_ylabel('Probability')
    ax2.set_title('Class Probabilities', fontsize=13)
    ax2.set_ylim(0, 1)
    ax2.grid(axis='y', alpha=0.3)

    plt.tight_layout()
    plt.show()
    print(f'Prediction: {predicted_digit} | Confidence: {confidence:.2f}%')
    # Run it
    predict_and_show('/content/WhatsApp Image 2026-06-01 at 9.57.37 AM.jpg')

```

output:

... Your Input → Predicted: 8
Confidence: 96.2%



... Your Input → Predicted: 8
Confidence: 96.2%

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 13

Design and Implementation of Artificial Neural Networks (ANN).

Name: Bushra Nazir Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Incomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
	SUB-TOTAL (PT)	7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
	SUB-TOTAL (CT)	1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
	SUB-TOTAL (AT)	2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) :

10

Marks Obtained (PO + CO + AO) :

Instructor